

Experimental Evaluation of SVM Kernels for Binary Image Classification

George Ioannidis, undergraduate student, ECE AUTH

I. INTRODUCTION

The purpose of this assignment is to develop, train and evaluate an SVM for the image classification problem of animals and vehicles, and to compare its performance with: a) an MLP with one hidden layer using Hinge Loss for optimization and b) the three baseline classifiers: Nearest Centroid (NC), 1-Nearest Neighbor (1-NN) and 3-Nearest Neighbor (3-NN).

The study was carried out using the CIFAR-10 dataset, which contains 60,000 color images of 32x32 pixels, of which 50,000 are used for training and 10,000 for testing, classified into ten different object categories.

II. DATA & PREPROCESSING

Reshaping & Flatten:

Initially, the data were loaded through the `cifar10.load_data()` function of the Keras library. Since the data are provided as tensors with dimensions (N, 32, 32, 3), where N is the number of images, they had to be transformed so as to be compatible with the algorithms of the Scikit-Learn library. For this reason, the reshape process was applied to the input data `x_train` and `x_test`, so that each image was converted into a one-dimensional feature vector of length 3072. In this way, the training matrix obtained the form (50000, 3072). At the same time, the target labels, `y_train` and `y_test`, were converted into a one-dimensional array through flattening.

Label Remapping:

Next, the original ten-class CIFAR-10 problem was reformulated as a binary classification problem by defining two super-classes: Vehicles and Animals. The Vehicles super-class includes the categories Airplane, Automobile, Ship and Truck, to which the label +1 was assigned (20k images in total). Correspondingly, the Animals super-class includes the remaining six categories (Bird, Cat, Deer, Dog, Frog, Horse), to which the label -1 was assigned (30k images in total).

Data Scaling:

In the final preprocessing stage, standardization of the input features was applied. Specifically, the `StandardScaler` transformation of Sklearn was used, which transforms each feature so that it has zero mean and unit variance, according to

the relation:

$$x' = \frac{x - \mu}{\sigma}$$

where μ and σ correspond to the mean value and the standard deviation of the feature, respectively. The transformation was fitted exclusively on the input features of the training set and was then applied to both the train set and the test set. Standardization is necessary for SVM training, because without scaling, features with a larger numerical scale can dominate the learning process, negatively affecting the calculation of the decision hyperplane and the generalization of the model.

III. SVM MODELS & EXPERIMENTAL PROCEDURE

Implementation & Acceleration:

To accelerate the experimental procedure, the GPU-optimized implementations of the CUMMLIB library were used. In particular, for the case of the linear kernel, the `cuml.svm.LinearSVC` class was selected in combination with a manual implementation of Grid Search, in order to achieve optimal computational performance. In contrast, for the non-linear kernels, training was based on the `cuml.svm.SVC` class, while parameter optimization was carried out through the `cuml.model_selection.GridSearchCV` class.

Kernel Study:

The study focused on investigating both the linear kernel: $K(x_i, x_j) = x_i^T x_j$ as well as the non-linear kernels RBF (Radial Basis Function): $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, $\gamma > 0$ and Polynomial: $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$ where: d is the degree of the polynomial, γ is the scale coefficient, and r is the constant term (`coef0`). For each kernel, different hyperparameter combinations were examined, aiming to analyze their effect on performance and model generalization.

Evaluation Methodology:

The evaluation of the different hyperparameter combinations was carried out through Grid Search, incorporating 5-Fold Cross-Validation in order to minimize training bias. For each examined hyperparameter combination, the mean accuracy was calculated and recorded both for the training sets and for the validation sets. The comparative evaluation of the different parameter combinations was based on the parallel examination

of these two accuracy metrics, with the aim of finding the best balance between high accuracy and avoiding overfitting.

Comparison Models:

To evaluate the performance of the SVMs, the experimental procedure included their comparison with four additional classifiers:

Nearest Centroid (NC): The Nearest Centroid classifier was implemented using the NearestCentroid class of the sklearn.neighbors library.

k-Nearest Neighbors (k-NN): The k-Nearest Neighbors classifier was implemented using the KNeighborsClassifier class of the sklearn.neighbors library. In the context of the study, the cases $k=1$ (1-NN) and $k=3$ (3-NN) were examined, in order to analyze the effect of the number of neighbors on the performance and generalization of the model.

Multi-Layer Perceptron (MLP): An MLP neural network with one hidden layer consisting of 256 neurons was developed. The model was implemented using the PyTorch framework. Before the beginning of neural network training, the data were standardized through StandardScaler. Then, the initial training set was divided into training and validation sets in an 80%-20% ratio, using a stratified split so that the class distribution would be preserved. The data were converted into tensors and organized into DataLoaders, with batch size 256 for the training set and 1024 for the validation and test sets. To stabilize training, Batch Normalization was applied to the hidden layer, while the model parameters were optimized with the Adam algorithm, using learning rate=0.001 and L2 regularization through weight decay = $1e-4$. Hinge Loss was selected as the cost function, in order to simulate the operation of an SVM within a neural-network environment and to make the comparison between the two approaches more direct.

IV. RESULTS & COMMENTARY

Hyperparameter Tuning:

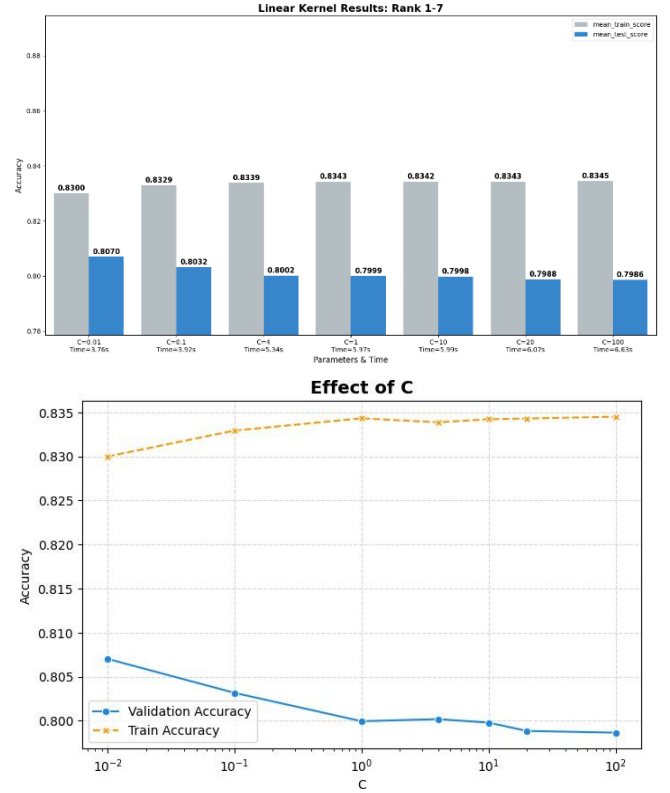
The procedure for selecting the best hyperparameters for all examined kernels followed an iterative search strategy. Initially, a broad search (loose grid search) was carried out, in order to identify the appropriate order of magnitude of the basic hyperparameters of each kernel. Then, based on the results of this phase, successive focused searches (fine grid searches) were applied in increasingly restricted areas of the hyperparameter space, until convergence toward the optimal combination was achieved.

Support Vector Machines (SVM):

1. Linear Kernel

Loose Grid Search:

In the first stage, the range of values of parameter $C \in [0.01, 0.1, 1, 4, 10, 20, 100]$ was examined:

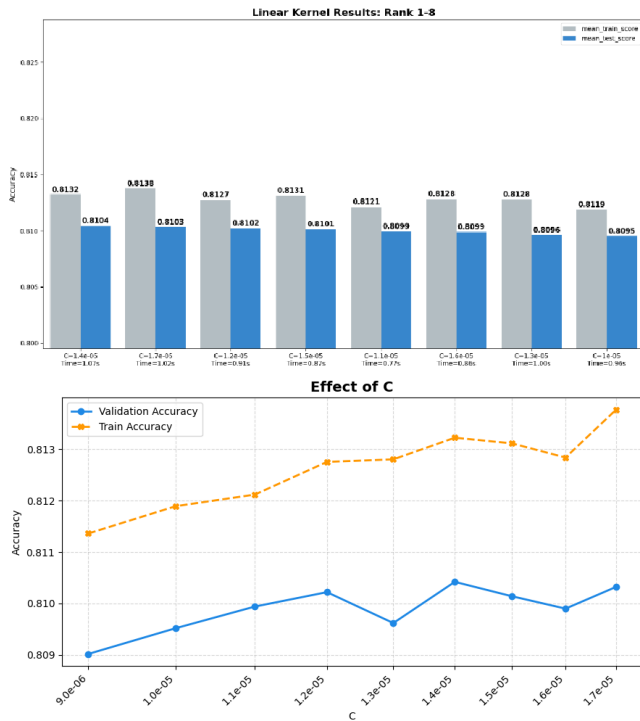


During this initial stage, the maximum validation accuracy was found for $C=0.01$, reaching 80.7%. As shown in the diagrams, a further increase of parameter C leads to a gradual decrease in accuracy on the validation set, while at the same time the train accuracy shows an increasing trend.

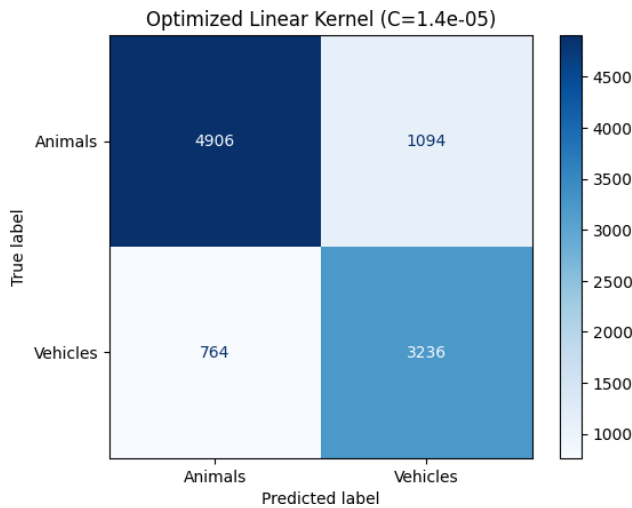
This behavior indicates that, for larger C values, the model penalizes classification errors more strictly, and as a result it adapts increasingly to the training data. Consequently, although higher accuracy is achieved on the training set, the generalization ability of the model decreases, which is an indication of overfitting.

Fine Grid Search:

Based on the above findings, the further investigation focused on values of parameter C smaller than the initially optimal value $C=0.01$. After successive focused searches, the search range was progressively restricted to very small C values, finally leading to the investigation of the region $C \in [9e-6, 1e-5, 1.1e-5, \dots, 1.7e-5]$



C=1.4e-5 was selected as the optimal hyperparameter, because, as shown by the graphs, it maximizes the validation accuracy. With this choice, the model achieved Test Accuracy: 81.42%. Although for C=1.7e-5 a marginally higher training accuracy is observed, this is not accompanied by a corresponding improvement on the validation set, indicating the beginning of a tendency toward overfitting. The Confusion Matrix for the optimal choice of C is presented in the following figure:



The matrix demonstrates the limitation of the linear kernel in capturing the non-linear structures of the data, which leads to increased confusion between the two classes. The larger number of errors in the Animals class is due to the class imbalance (60%-40%). Nevertheless, the application of weight balancing ensured almost equivalent recall for Animals and

Vehicles (81.8% for Animals and 80.9% for Vehicles), indicating the absence of bias toward the majority class.

Examples of Correct and Incorrect Classification - Linear Kernel



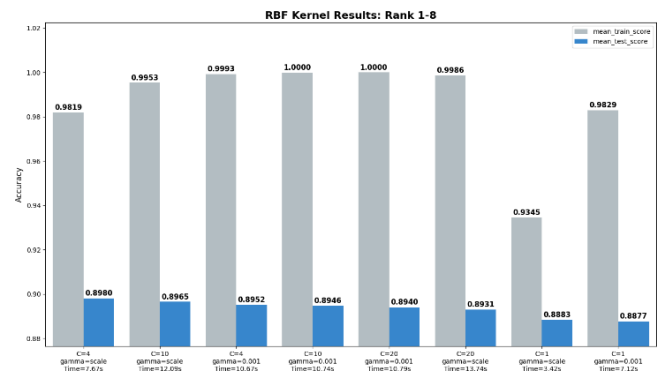
The above image presents characteristic examples of correct and incorrect classification. Wrong predictions are mainly observed in images where the visual characteristics of the two classes overlap, such as strong edges and noise, leading to confusion between animals and vehicles. These examples highlight the difficulty of the problem and explain the need for non-linear kernels.

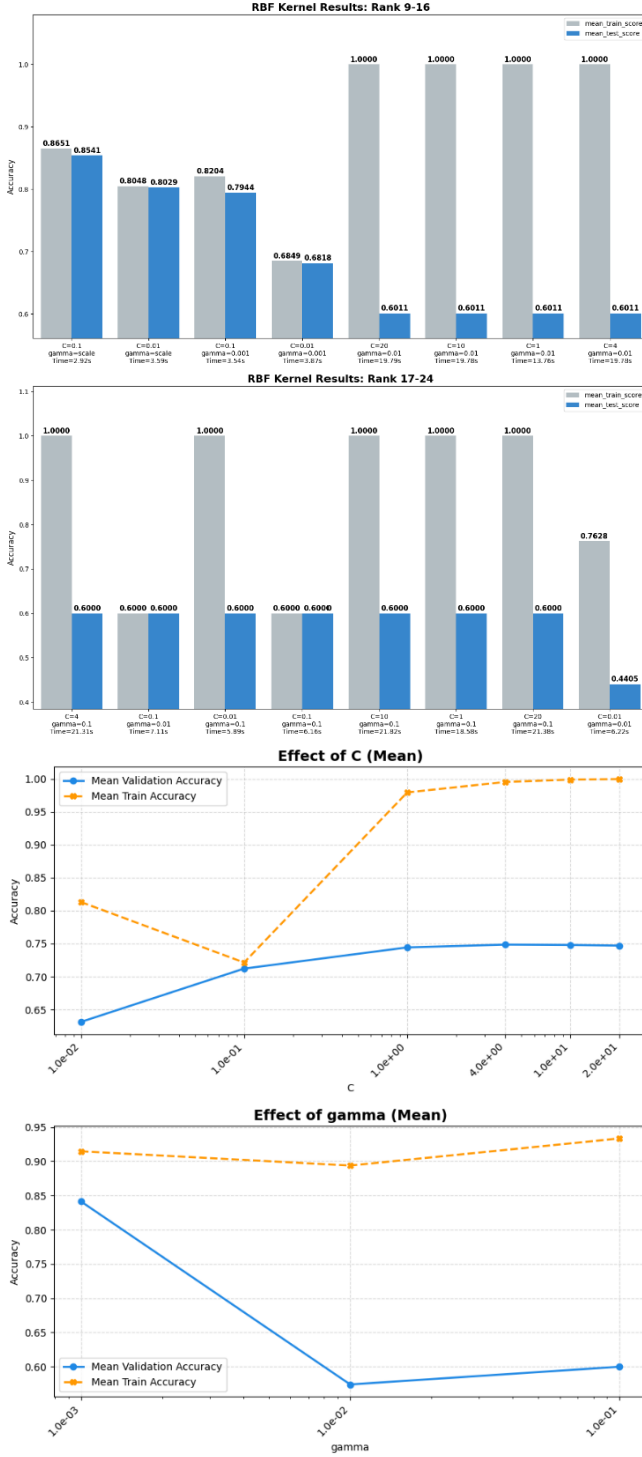
2. RBF Kernel

Loose Grid Search:

During the initial stage of the loose grid search for the RBF kernel, different combinations of the hyperparameters C and gamma were examined. Specifically, parameter C took values from the set [0.01, 0.1, 1, 4, 10, 20], while gamma was examined for the values [scale, 0.001, 0.01, 0.1], where gamma=scale corresponds to the calculation of gamma based on the variance of the features, given by the formula:

$$\text{gamma} = \frac{1}{n_{\text{features}} * \text{VAR}(X)} = \frac{1}{3072 * 1} \approx 3.25e - 4$$





It is observed that the best performances on the validation set are achieved for values of parameter $C \in [4, 10]$ combined with small values of gamma, especially $\text{gamma}=0.001$ and $\text{gamma}=\text{scale}$. These combinations lead to high validation accuracy, while also maintaining a controlled deviation between train and validation accuracy, indicating good generalization ability. Therefore, the search will be restricted near these hyperparameter combinations.

For larger values of parameter C (4, 10, 20), the training accuracy approaches almost 100%, while the validation accuracy reaches saturation without any further substantial

improvement, indicating limited benefit from further increasing parameter C .

In contrast, for low C values, such as $C=0.1$ or $C=0.01$, a drop in accuracy is observed both on the training and on the validation set (from 89% to 80-85% and 68%). This behavior indicates underfitting, which is due to the fact that, for such small C values, the SVM shows high tolerance to classification errors in order to maximize the margin. This leads to an overly simplified decision boundary, which cannot adapt to the non-linearly separable structure of the data.

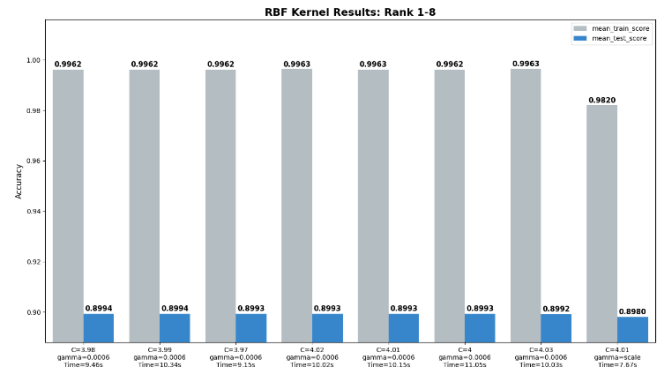
At the same time, for high gamma values (0.1, 0.01), the model achieves 100% training accuracy, but this is accompanied by a significant drop in validation accuracy (60%). Thus, an overfitting phenomenon is observed, as the RBF kernel becomes excessively local and adapts strongly to the training data at the expense of generalization.

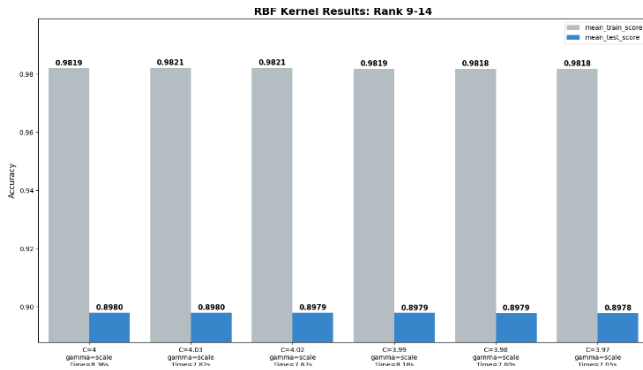
In addition, the combination of low C values with high gamma values leads to very poor performance on both the training and validation sets, which also confirms the presence of strong underfitting, as the kernel tendency for local adaptation (high gamma) conflicts with the excessive restrictions imposed by the small value of C .

Finally, it is worth noting the significant variation in training times. It is observed that increasing parameter C increases the computational cost, because the model attempts to satisfy stricter margin constraints. Specifically, for $\text{gamma}=\text{scale}$, the mean time per fold increases from 3.1s for $C=0.1$ to 15.1s for $C=100$. However, the highest mean times, 20.3s (from 5-fold cross validation), are recorded in cases where gamma takes high values (such as 0.1), for which the model shows poor generalization. Since a large percentage of samples are defined as Support Vectors, the model complexity increases significantly. This increase burdens the optimization process, as more iterations are required for convergence. $\approx \approx \approx$

Fine Grid Search:

The investigation focused on values of parameter C around 4, which achieves the best performance. The final search interval became $C \in [3.97, 3.98, 3.99, 4, 4.01, 4.02, 4.03]$ and $\text{gamma} \in [0.0006, \text{scale}]$.



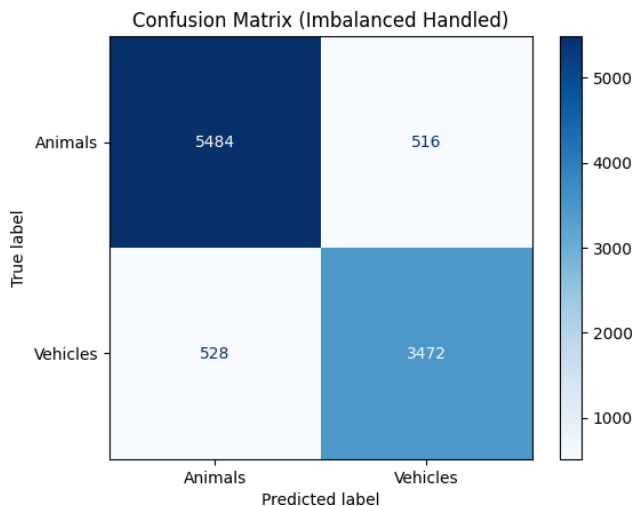


Although the combinations $C=3.98$ and $C=3.99$ with $\gamma=0.0006$ achieve marginally the highest validation accuracy, they are accompanied by very high train accuracy ($\approx 99.6\%$), as also happens with the other C choices using the same γ . However, these choices require a significantly larger number of Support Vectors ($\approx 25,800$), which indicates increased model complexity and a tendency toward overfitting.

In contrast, the choice $\gamma=scale$ leads to slightly lower validation accuracy (difference $\approx 0.14\%$), but it presents a clearly smaller training-validation gap and a significantly reduced number of support vectors (from $\approx 25,800$ to 20,458). This behavior indicates better generalization ability and a smoother decision boundary.

For this reason, the combination $C=4.01$ and $\gamma=scale$ is selected as the final optimal choice for the RBF kernel, offering the best balance between accuracy and generalization.

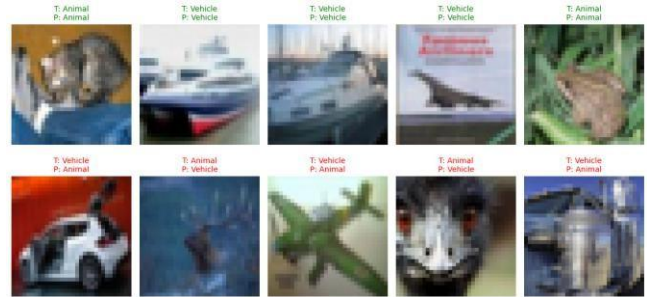
The final evaluation on the test set confirmed the superiority of the $\gamma=scale$ choice ($C=4.01$), since the reduction of 5,400 Support Vectors did not negatively affect the performance of the model. Although a small differentiation was observed in the behavior of the models per class (with $\gamma=0.0006$ slightly favoring the Animals class and $\gamma=scale$ favoring the Vehicles class), the overall performance of the model with $\gamma=scale$ was superior, achieving accuracy $\approx 90.27\%$ versus 90.25% .



The Confusion Matrix of the optimal RBF model shows a balanced distribution of errors between the two classes, without any evident bias toward either the majority or the minority category. The model achieves a high rate of correct classifications both for Animals and for Vehicles, significantly limiting both false positives and false negatives. This behavior confirms the increased generalization ability of the RBF kernel, which, unlike the linear kernel, can effectively model the non-linear structure of the data. As a result, the RBF SVM clearly outperforms the Linear SVM, achieving a more reliable and stable separation of the two categories on the test set.

Examples of Correct and Incorrect Classification

RBF Kernel

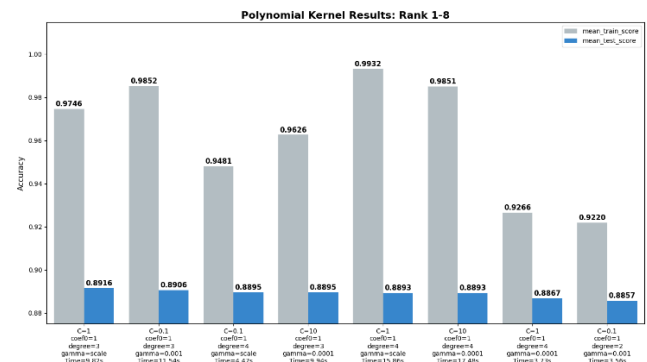


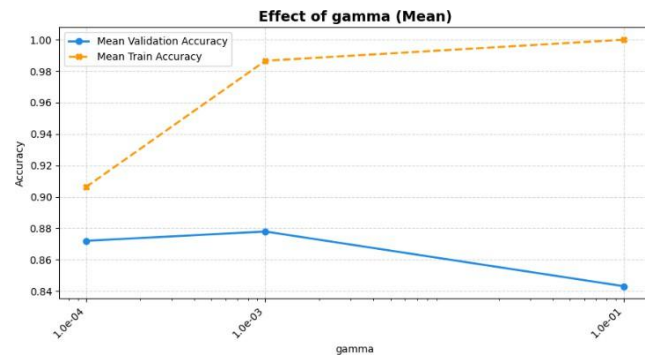
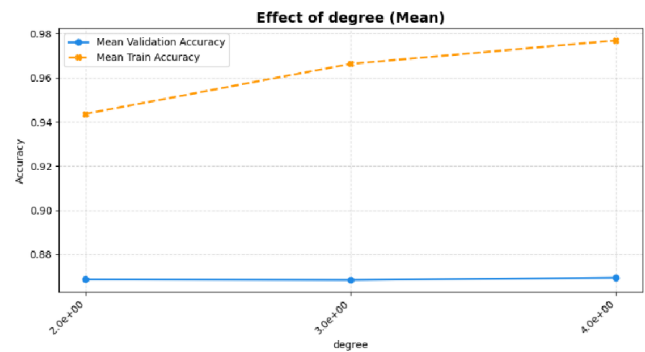
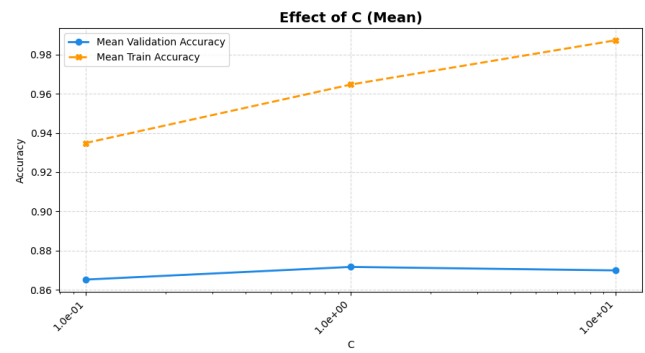
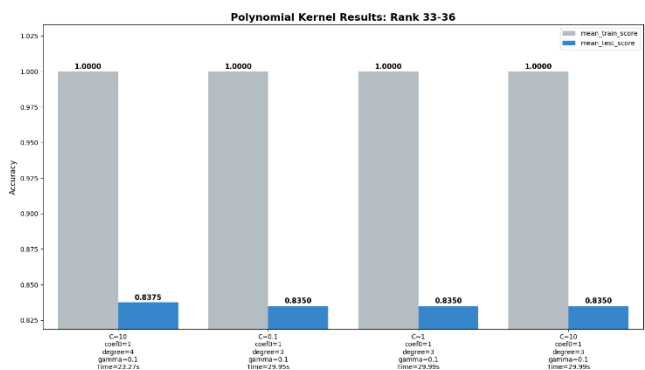
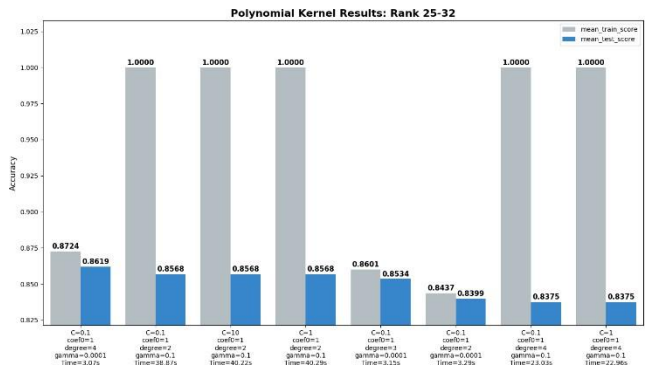
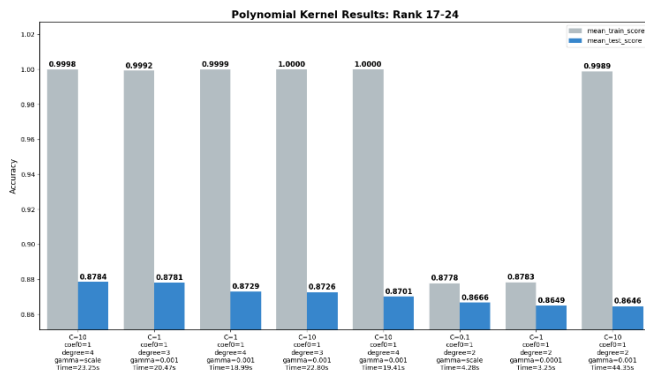
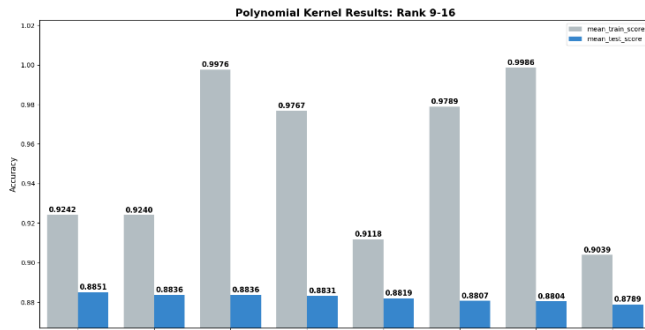
These examples show that the model errors are not random, but occur in cases where the distinction between the two super-classes is inherently ambiguous. Despite these individual failures, the overall behavior of the model confirms that it effectively captures most of the discriminative features of the data.

3. Polynomial Kernel

Loose Grid Search:

During the initial stage of the loose grid search for the polynomial kernel, combinations of the hyperparameters C , γ and the polynomial degree d were examined. The set of examined values was defined as follows: $C \in [0.1, 1, 10]$, $d \in [2, 3, 4]$, $\gamma \in [scale, 0.0001, 0.001, 0.1]$, while the constant term $coef0$ was equal to 1 in every case.





From the loose grid search results, it is observed that the highest accuracy on the validation set is achieved for $C=1$, $\text{degree}=3$ and $\text{gamma}=\text{scale}$, with validation accuracy of approximately 89.2%. Very close performances are also recorded for $\text{gamma}=0.001$, indicating that the optimal hyperparameter region lies around $C \approx 1$, $\text{degree}=3$ and small values of gamma .

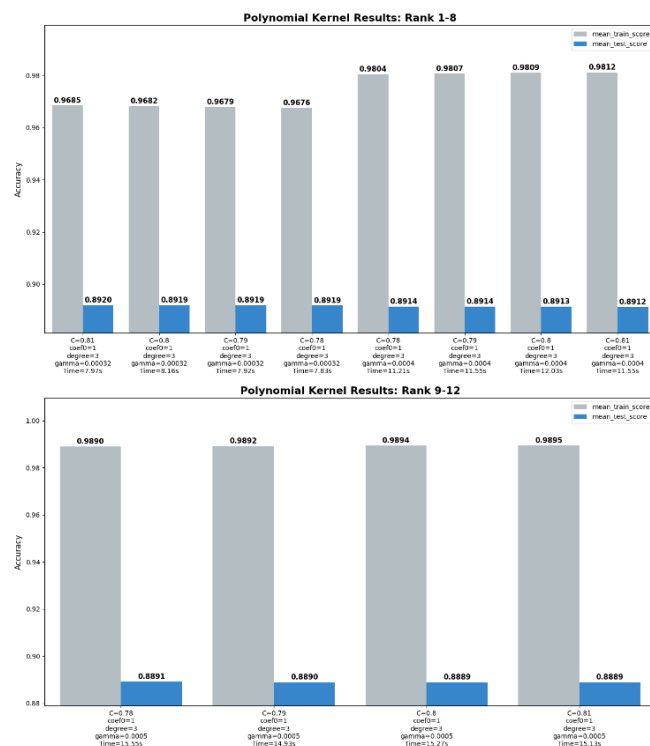
In contrast, a further increase in parameter C and/or in the polynomial degree mainly leads to an increase in training accuracy, without a corresponding improvement on the validation set, indicating a tendency toward overfitting. This phenomenon is also clearly reflected in the Effect of degree (Mean) diagram, where increasing the degree from 2 to 3 and 4 causes a significant rise in training accuracy, while validation accuracy remains almost unchanged.

This finding shows that increasing the complexity of the polynomial kernel through degree d does not translate into a substantial improvement in generalization. Instead, it mainly strengthens the ability of the model to fit the training data, increasing the risk of overfitting. Therefore, medium polynomial degrees ($\text{degree}=2$ or 3) are preferable and formed the basis for restricting the hyperparameter search in the next stage of fine grid search.

Regarding training time, a noteworthy finding is observed for parameter $C=100$. Specifically, the 2nd-degree polynomial requires on average 66 seconds (1.1 minutes) per fold, while the 3rd-degree polynomial completes the process in 29 seconds. The fact that the simpler model takes twice as long is due to the inability of the 2nd-degree model to achieve the required separation with a small number of margin violations. As a result, the optimization process has difficulty converging and requires more iterations. In contrast, the 3rd-degree polynomial model is more flexible, achieves the required separation more easily and therefore converges faster, despite its increased theoretical complexity.

Fine Grid Search:

The investigation focused on values of C around 0.8, where the best performance was found. The final search interval was set as $C \in [0.78, 0.79, 0.8, 0.81]$ and $\gamma \in [\text{scale}, 0.0004, 0.0005]$, while the parameters degree and coef0 remained fixed at the values 3 and 1 respectively.

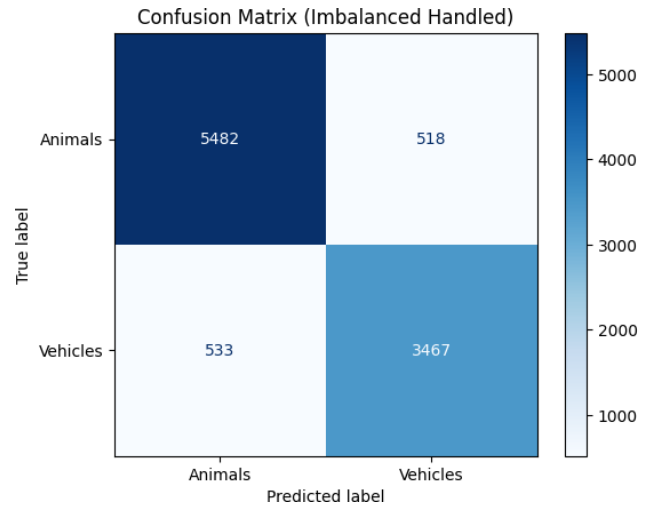


As shown by the diagrams, the individual changes of parameters C and γ within the examined region lead to very small differences in validation accuracy, which essentially remains stable in the range 89.1%-89.2%. In contrast, train accuracy increases for larger γ values, without however improving the generalization ability of the model.

The choice $\gamma=\text{scale}$ (0.000325) is considered the best with $C=0.81$, using 17,474 Support Vectors, because it presents a smaller gap between train accuracy (96%) and validation accuracy (89.5%) compared with $\gamma=0.0004$ (train 98%).

Both models achieved an identical accuracy score on the test set ($\approx 89.47\%$). The fact that the model with $\gamma=0.0004$ showed a higher train score without a

corresponding improvement on the test set proves that the additional success rate in training is due to greater overfitting and not to substantial learning.



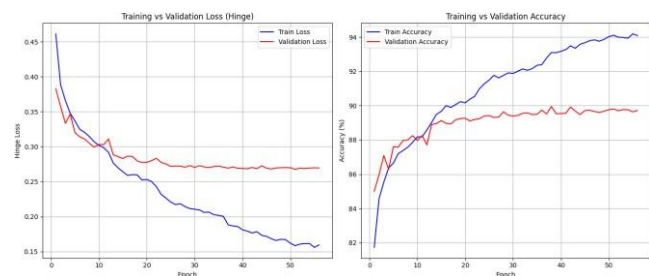
Compared with the RBF kernel, the polynomial kernel shows a slightly greater, but not substantial, differentiation between False Positive and False Negative errors. This deviation remains small in scale and does not indicate strong bias toward any class, confirming that both models maintain balanced behavior in terms of generalization.

Comparison of Misclassification Examples between Polynomial and RBF Kernel



The examples where the Polynomial Kernel fails but the RBF classifies correctly (and vice versa) show that, although the two non-linear kernels achieve similar overall accuracy, they do not learn exactly the same decision boundaries.

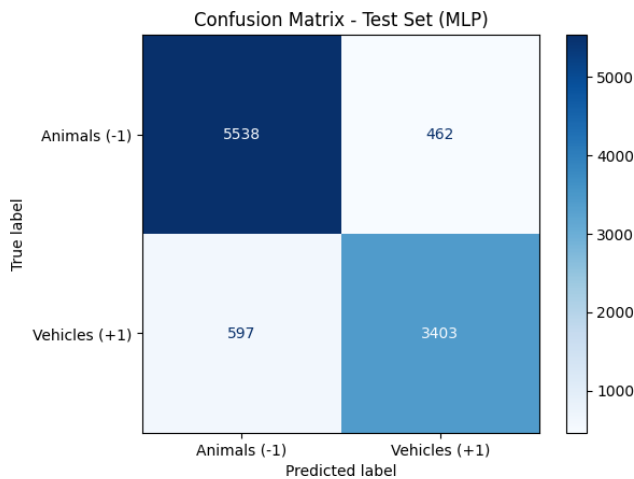
Multi-Layer Perceptron (MLP)



The training of the Neural Network, as shown in the diagrams, presented relatively smooth convergence. The value of the Hinge Loss cost function on the training set decreased steadily, while the corresponding value on the validation set stabilized after the first epochs, activating Early Stopping to avoid overfitting.

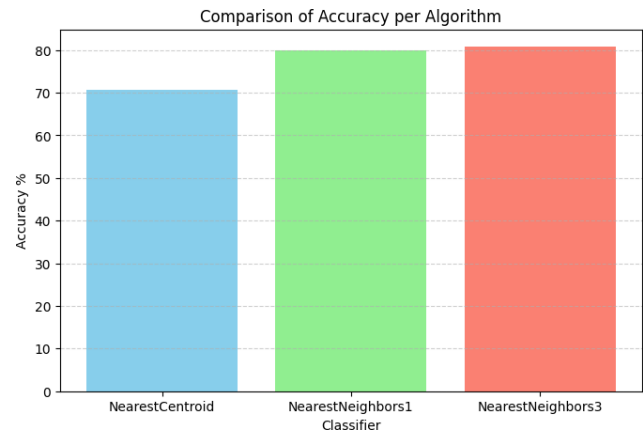
Specifically, the accuracy on the training set reached approximately 94%, while on the validation set it stabilized close to 89.7%.

The final evaluation on the Test Set gave an overall Accuracy of 89.40%. This performance makes the MLP competitive in relation to the SVM models, proving that a shallow neural network with one hidden layer and Hinge Loss can generalize satisfactorily in this specific problem. <<>>



Baseline Classifiers

macro avg			
Classifier	precision	recall	f1-score
Nearest Centroid	0.70	0.70	0.70
NearestNeighbors(k=1)	0.80	0.78	0.78
NearestNeighbors(k=3)	0.81	0.78	0.79



Nearest Centroid (NC):

The model recorded the lowest performance, with Test Accuracy 71%.

k-Nearest Neighbors (k-NN):

The k-NN classifier achieved significantly better results. Specifically, for k=1 the accuracy reached 80%, while for k=3 it improved slightly to 81%. This improvement is due to the fact that 3-NN is less sensitive to outliers than 1-NN.

Nevertheless, their performance remained approximately 9-10% lower than the best SVM and MLP models, proving the need for stronger classifiers.

V. CONCLUSIONS AND COMPARISON

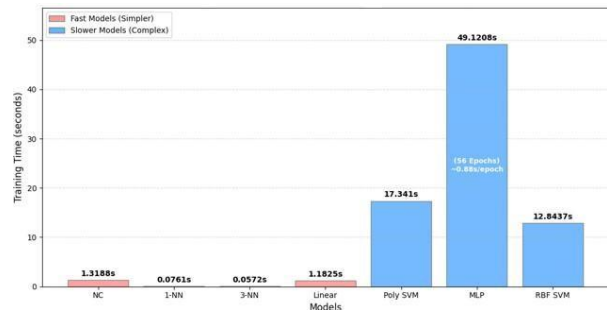
The comparative evaluation of the classification algorithms on the Animals vs Vehicles classification task led to the following conclusions:

- Superiority of Non-Linear Kernels:** Regarding the comparison of the different SVM versions, it was shown that, for this specific classification task, the non-linear kernels (RBF and Polynomial) achieved significantly higher results than the Linear kernel. Specifically, the RBF and Polynomial SVM achieved Test Set accuracies of 90.27% and 89.49%, respectively, compared with only 81.42% for the Linear SVM. This gap (approximately 9%) proves that the image features are not linearly separable in the original space. Therefore, the creation of more complex, non-linear decision boundaries is necessary for the effective classification of the data.
- Selection of RBF as the Best Model:** The SVM model with RBF kernel emerged as the best model for solving this classification problem. With optimal parameters $C=4.01$ and $\gamma=\text{scale}$, it achieved the highest generalization ability, 90.27%. This superiority is due to the property of the RBF kernel to project the data into an infinite-dimensional space, allowing the classes to be separated by complex, non-linear boundaries. At the same time, with this specific hyperparameter combination, overfitting was avoided by reducing the number of Support Vectors, ensuring high accuracy combined with optimal generalization ability.

3. **MLP vs SVM:** The simple Neural Network (MLP) with only one hidden layer and Hinge Loss achieved 89.30%, approaching the performance of the non-linear SVMs. This proves that, even with a simple architecture, neural networks can generalize satisfactorily when they are properly tuned through the appropriate techniques (BatchNorm, Dropout, Early Stopping, etc.).
4. **Linear SVM vs k-NN:** The 3-NN model achieved 81.00% accuracy, approaching the performance of the Linear SVM (81.42%). This finding further confirms the weakness of the Linear Kernel in this specific problem, since the more advanced SVM classifier did not manage to substantially outperform the naive model based simply on Euclidean distance.

Summary Table of Results

	Model	Best Hyperparameters	Test Accuracy
1	SVM (RBF Kernel)	$C=4.01, \gamma=0.000325$	90.27%
2	SVM (Polynomial)	$C=0.81, d=3, \gamma=0.000325$	89.49%
3	MLP (Hinge Loss)	1 Hidden Layer, 256 Neurons, Dropout 0.45	89.40%
4	SVM (Linear)	$C=1.4e-5$	81.42%
5	k-NN (k=3)	k=3, Euclidean Dist.	81.00%
6	k-NN (k=1)	k=1, Euclidean Dist.	80.00%
7	Nearest Centroid	-	71.00%



The training-time comparison diagram clearly highlights the trade-off between computational complexity and model accuracy. The baseline classifiers present the lowest training times. Nearest Centroid (NC) requires about 1.3s, a time that remains significantly lower compared with the more complex models, while k-NN (1-NN and 3-NN) show almost zero training time ($<0.1s$), because they do not include a learning procedure. Despite their low computational requirements, these models lag significantly in accuracy.

In contrast, the SVM models present increased computational cost. The Linear SVM maintains a low training time, while the non-linear kernels show a significant burden. The Polynomial SVM requires about 17.3s, while the RBF SVM, despite its high accuracy, maintains a more controlled training time ($\sim 12.8s$), which makes it a more efficient choice compared with the polynomial kernel.

The MLP presents the highest total training time ($\sim 49.1s$), because its training is performed iteratively over multiple epochs (56 epochs). Although this time burden is noticeable, the MLP remains competitive in terms of accuracy, confirming that neural networks need more training time but perform well on new data.

Overall, the SVM with RBF kernel achieves the best balance between training time and performance, combining relatively low computational cost with high accuracy and strong generalization ability.

